RetinAI

Documentation Technique Détaillée

Guide complet du pipeline de Computer Vision
pour la détection de rétinopathie diabétique

Pipeline ML · TensorFlow · MLflow · Docker

Version 1.0 — Mars 2026

Table des matières

1 Présentation du projet **3**
- 1.1 Contexte médical . . . 3
- 1.2 Objectif du pipeline . . . 3
- 1.3 Stack technique . . . 4

2 Architecture du projet **4**
- 2.1 Structure des fichiers . . . 4

3 Module de prétraitement (`src/data/`) **4**
- 3.1 Classe `RetinalImagePreprocessor` . . . 5
  - 3.1.1 Étape 1 : Extraction de la ROI circulaire . . . 5
  - 3.1.2 Étape 2 : Normalisation Ben-Graham . . . 5
  - 3.1.3 Étape 3 : CLAHE . . . 5
  - 3.1.4 Étape 4 : Resize et normalisation . . . 5
- 3.2 Classe `RetinalDataset` . . . 6
  - 3.2.1 Gestion du déséquilibre de classes . . . 6
  - 3.2.2 Split stratifié . . . 6
  - 3.2.3 Augmentations . . . 6

4 Architecture du modèle (`src/models/`) **6**
- 4.1 Vue d'ensemble . . . 6
- 4.2 Backbone : EfficientNetV2-S . . . 7
- 4.3 Module d'attention spatiale (CBAM) . . . 7
- 4.4 Tête de classification . . . 7
- 4.5 Stratégie de fine-tuning en 2 phases . . . 8
- 4.6 Fonctions de perte . . . 8
  - 4.6.1 Focal Loss pondérée . . . 8
  - 4.6.2 QWK Loss . . . 8

5 Entraînement et suivi MLflow (`src/models/train.py`) **8**
- 5.1 Pipeline d'entraînement . . . 8
- 5.2 Callbacks Keras . . . 9
- 5.3 Suivi MLflow . . . 9

6 Optimisation du modèle (`src/models/optimize.py`) **9**
- 6.1 Pruning structurel . . . 9
- 6.2 Quantification . . . 10
- 6.3 Test-Time Augmentation (TTA) . . . 10
- 6.4 Export ONNX . . . 10

7 Évaluation et explicabilité (`src/evaluation/`) 10
7.1 Métriques complètes . . . 10
7.2 Résultats obtenus . . . 11
7.3 Explicabilité : Grad-CAM . . . 11
7.4 Calibration de confiance (ECE) . . . 11
7.5 Seuils de triage clinique . . . 12
7.6 Analyse des erreurs . . . 12

8 API d'inférence (`src/api/`) 12
8.1 Endpoints FastAPI . . . 12
8.2 Format de réponse . . . 12
8.3 Modèle TFLite en production . . . 13

9 Déploiement Docker 13
9.1 Dockerfile multi-stage . . . 13
9.2 Docker Compose . . . 13

10 Tests et qualité de code 13
10.1 Tests unitaires . . . 13
10.1.1 `test_preprocessing.py` . . . 13
10.1.2 `test_model.py` . . . 14
10.2 Commandes Makefile . . . 14

11 Configuration et utilitaires 14
11.1 Fichier de configuration YAML . . . 14
11.2 Logging structuré . . . 14

1 Présentation du projet

1.1 Contexte médical

La **rétinopathie diabétique** (RD) est une complication micro-vasculaire du diabète qui affecte la rétine. C'est la première cause de cécité chez les adultes en âge de travailler dans les pays industrialisés, touchant plus de 100 millions de personnes dans le monde.

Le dépistage repose sur l'analyse d'images de **fond d'œil** (photographies de la rétine prises avec un rétinographe). Un ophtalmologue examine ces images à la recherche de lésions caractéristiques :

Grade	Niveau	Description des lésions
0	**No DR**	Aucune anomalie détectable sur la rétine.
1	**Mild**	Présence de *micro-anévrismes* uniquement (petites dilatations des capillaires rétiniens).
2	**Moderate**	Micro-anévrismes + *hémorragies intra-rétiniennes*, *exsudats durs* (dépôts lipidiques) ou anomalies veineuses.
3	**Severe**	Hémorragies dans les 4 quadrants, anomalies veineuses dans 2+ quadrants, ou anomalies micro-vasculaires intra-rétiniennes.
4	**Proliferative**	*Néovascularisation* (formation anarchique de nouveaux vaisseaux) et/ou hémorragie du vitré. Risque immédiat de cécité.

1.2 Objectif du pipeline

RetinAI automatise l'analyse des images rétiniennes pour **assister** (et non remplacer) les radiologues. Le pipeline prend en entrée une image de fond d'œil et produit en sortie :

1. Un **grade de sévérité** (0 à 4).
2. Un **score de confiance** calibré.
3. Une **carte de chaleur** (Grad-CAM) montrant les zones d'attention du modèle.
4. Une **recommandation clinique** (fréquence de suivi, nécessité de référer).

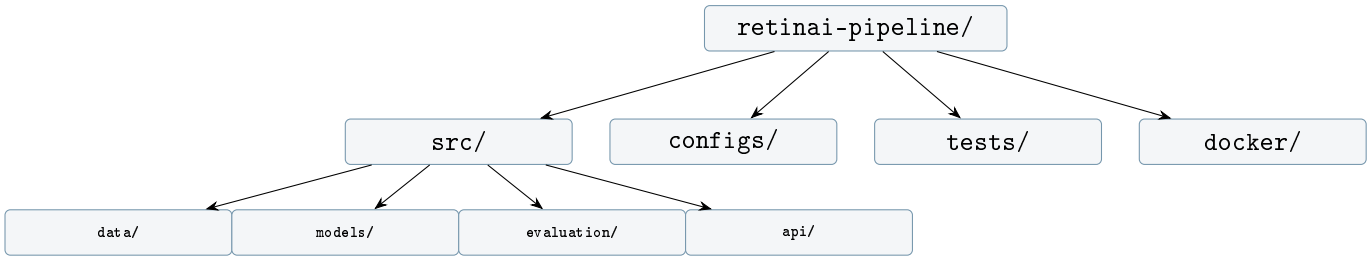
1.3 Stack technique

Composant	Technologie	Rôle
Langage	Python 3.10+	Langage principal
Framework ML	TensorFlow 2.15	Entraînement et inférence
Traitement d'images	OpenCV (cv2)	Preprocessing des images
Données	Pandas, NumPy	Manipulation et analyse des données
Expériences	MLflow	Tracking des runs, métriques, modèles
Conteneurisation	Docker, Docker Compose	Déploiement reproductible
API	FastAPI, Uvicorn	Service d'inférence REST
Tests	pytest	Tests unitaires et d'intégration

2 Architecture du projet

2.1 Structure des fichiers

Le projet suit une organisation modulaire qui sépare clairement les responsabilités :



- `src/data/` — Chargement, prétraitement et augmentation des images.
- `src/models/` — Architecture du réseau, fonctions de perte, entraînement et optimisation.
- `src/evaluation/` — Métriques, Grad-CAM, calibration, rapport clinique.
- `src/api/` — Endpoint FastAPI pour le service d'inférence.
- `src/utils/` — Configuration centralisée et logging structuré.
- `configs/` — Fichier YAML de paramétrage.
- `tests/` — Tests unitaires (preprocessing, architecture, losses).
- `docker/` — Dockerfile multi-stage pour le déploiement.

3 Module de prétraitement (`src/data/`)

3.1 Classe `RetinalImagePreprocessor`

Le préprocesseur implémente un pipeline en 4 étapes séquentielles appliquées à chaque image.

3.1.1 Étape 1 : Extraction de la ROI circulaire

Les images de fond d'œil contiennent typiquement un disque circulaire (la rétine visible) sur un fond noir. Cette étape détecte automatiquement ce disque et crop l'image autour de celui-ci.

Comment ça marche

1. L'image est convertie en niveaux de gris.
2. Un seuillage binaire ($t = 15$) sépare le disque du fond noir.
3. La détection de contours identifie le plus grand contour (le disque).
4. Le cercle englobant minimal est calculé (centre + rayon).
5. L'image est croppée avec une marge de 5% et un masque circulaire est appliqué.

Pourquoi c'est important : Sans cette étape, le fond noir occupe une grande partie de l'image et fausse les statistiques de normalisation (moyennes, écarts-types). De plus, le réseau gaspille de la capacité à traiter des pixels non informatifs.

3.1.2 Étape 2 : Normalisation Ben-Graham

Les rétinographes produisent des images avec des illuminations très variables (selon l'appareil, le patient, la dilatation pupillaire). La normalisation Ben-Graham uniformise l'éclairage en soustrayant la composante basse fréquence.

En pratique : L'image est multipliée par 4, le flou gaussien (avec $\sigma = 10$) est soustrait (multiplié par 4 également), et un offset de 128 est ajouté pour recentrer les valeurs. Le résultat conserve uniquement les détails haute fréquence (vaisseaux, lésions).

3.1.3 Étape 3 : CLAHE

Le CLAHE rehausse le contraste *localement* en travaillant sur le canal L (luminance) de l'espace colorimétrique LAB. L'image est découpée en tuiles de 8×8, chaque tuile subit une égalisation d'histogramme avec un seuil d'écrêtage (`clip_limit` $= 2.0$), puis les résultats sont interpolés pour éviter les artefacts de blocs.

Pourquoi LAB et pas RGB : Travailler en LAB permet de modifier le contraste (canal L) sans altérer les informations de couleur (canaux a et b). Cela préserve les nuances chromatiques des lésions (hémorragies rouges, exsudats jaunes).

3.1.4 Étape 4 : Resize et normalisation

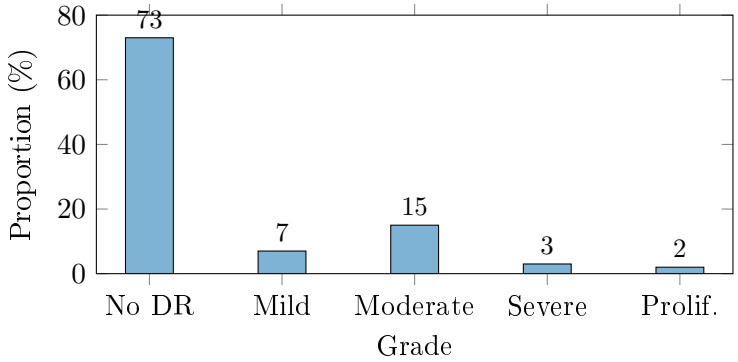
L'image est redimensionnée à 512×512 pixels avec préservation du ratio d'aspect (padding noir si nécessaire), puis les valeurs sont normalisées dans $[0, 1]$.

3.2 Classe `RetinalDataset`

Ce module gère le chargement des données, le split stratifié et les augmentations.

3.2.1 Gestion du déséquilibre de classes

La distribution typique d'un dataset de rétinopathie est très déséquilibrée :



Le module calcule automatiquement des poids de classe inversement proportionnels à la fréquence : $\alpha_c = \frac{N}{K \cdot N_c}$. Ainsi, les classes rares (Severe, Proliferative) reçoivent un poids plus élevé dans la fonction de perte.

3.2.2 Split stratifié

Les données sont divisées en 3 sous-ensembles (75% train, 15% validation, 10% test) en utilisant un split **stratifié** : la distribution des grades est préservée dans chaque sous-ensemble. C'est essentiel car un split aléatoire pourrait placer zéro échantillon du grade 4 dans le set de validation.

3.2.3 Augmentations

Le pipeline implémente deux niveaux d'augmentation :

1. **Augmentations pixel-level** : rotations (0, 90, 180, 270), flips horizontal/vertical, ajustements légers de luminosité, contraste et saturation.
2. **Augmentations batch-level** : MixUp (mélange convexe de paires d'images) et CutMix (remplacement de patches rectangulaires).

Le pipeline `tf.data` est optimisé avec le prefetching automatique, le chargement parallèle et le shuffling par buffer.

4 Architecture du modèle (`src/models/`)

4.1 Vue d'ensemble

4.2 Backbone : EfficientNetV2-S

EfficientNetV2 est une famille de réseaux convolutifs conçus pour optimiser conjointement la vitesse d'entraînement et la précision. La version S (Small) offre un bon compromis entre performance et coût computationnel.

Pourquoi EfficientNetV2 ?

- Pré-entraîné sur ImageNet (transfert de représentations visuelles génériques).
- Utilise des blocs Fused-MBConv dans les premières couches (plus rapides sur GPU).
- Intègre des modules Squeeze-and-Excitation pour le recalibrage des canaux.
- Scaling composé (profondeur, largeur, résolution) pour une efficacité optimale.

Le backbone prend en entrée une image $512 \times 512 \times 3$ et produit des feature maps de dimension $16 \times 16 \times 1280$.

4.3 Module d'attention spatiale (CBAM)

Entre le backbone et le classifieur, un module CBAM (Convolutional Block Attention Module) est inséré. Il se compose de deux sous-modules séquentiels :

1. **Attention par canal :** Apprend quels canaux (i.e., quels types de features) sont les plus importants en utilisant un Global Average Pooling et un Global Max Pooling suivis d'un MLP partagé. Produit un vecteur de poids $\in \mathbb{R}^C$.
2. **Attention spatiale :** Apprend quelles positions spatiales sont les plus informatives en agrégeant les feature maps le long de la dimension des canaux (avg + max), puis en appliquant une convolution 7×7.

Intérêt pour l'imagerie rétinienne : Les lésions (micro-anévrismes de quelques pixels, hémorragies localisées) sont de petites structures noyées dans une image de grande taille. Le module d'attention permet au réseau de se concentrer sur ces zones discriminantes plutôt que de traiter uniformément l'image entière.

4.4 Tête de classification

Après l'attention spatiale, les feature maps sont transformées en un vecteur de probabilités :

Couche	Détails
Global Average Pooling	$16 \times 16 \times 1280 \rightarrow 1280$
Batch Normalization	Stabilisation et régularisation
Dropout (40%)	Prévention de l'overfitting
Dense (256, ReLU)	Couche cachée non-linéaire
Dropout (20%)	Régularisation additionnelle
Dense (5, Softmax)	Probabilités sur les 5 grades

4.5 Stratégie de fine-tuning en 2 phases

Pourquoi deux phases ?

Si l'on dégèle immédiatement le backbone, les gradients provenant du classifieur (initialisé aléatoirement) peuvent détruire les représentations pré-apprises. La stratégie en deux phases garantit que le classifieur est d'abord "stabilisé" avant d'ajuster finement le backbone.

Phase 1 – Warmup (5 epochs) Le backbone est complètement gelé. Seules les couches de la tête de classification (Attention + Dense) sont entraînées avec un learning rate élevé (10^{-3}). Le classifieur apprend à mapper les features existantes vers les bonnes classes.

Phase 2 – Fine-tuning (30 epochs) Les derniers blocs du backbone (à partir de `block6a`) sont dégelés. Un learning rate réduit (10^{-4}) est utilisé pour ajuster finement les features de haut niveau tout en préservant les représentations de bas niveau (contours, textures).

4.6 Fonctions de perte

4.6.1 Focal Loss pondérée

La fonction de perte principale combine trois mécanismes :

1. **Focal modulation** ($\gamma = 2$) : Réduit la contribution des exemples faciles (bien classés) et force le modèle à se concentrer sur les cas difficiles.
2. **Pondération par classe** : Compense le déséquilibre naturel du dataset en attribuant des poids plus élevés aux classes rares.
3. **Label smoothing** ($\varepsilon = 0.05$) : Empêche le modèle d'être trop confiant, ce qui améliore la calibration.

4.6.2 QWK Loss

En complément, une loss basée sur le Quadratic Weighted Kappa optimise directement la métrique de référence. Sa particularité est de pénaliser les erreurs proportionnellement à la *distance* entre les grades : confondre le grade 0 avec le grade 4 est 16 fois plus coûteux que confondre le grade 0 avec le grade 1.

5 Entraînement et suivi MLflow (`src/models/train.py`)

5.1 Pipeline d'entraînement

La classe `TrainingPipeline` orchestre l'ensemble du processus :

1. Chargement de la configuration YAML.
2. Configuration du GPU (memory growth pour éviter les erreurs OOM).
3. Chargement et split stratifié des données.
4. Construction du modèle et compilation avec AdamW.

5. Phase 1 (warmup) avec backbone gelé.
6. Phase 2 (fine-tuning) avec backbone partiellement dégelé.
7. Sauvegarde du meilleur modèle et des artefacts.

5.2 Callbacks Keras

Callback	Rôle
MLflowCallback	Log en temps réel de toutes les métriques et du learning rate dans MLflow.
EarlyStopping	Arrête l'entraînement si le QWK de validation ne s'améliore pas pendant 10 epochs et restaure les meilleurs poids.
ReduceLROnPlateau	Divise le learning rate par 2 quand la loss de validation stagne pendant 5 epochs.
ModelCheckpoint	Sauvegarde automatique du modèle ayant le meilleur QWK sur la validation.

5.3 Suivi MLflow

MLflow est un outil open-source de gestion du cycle de vie ML. Dans RetinAI, il est utilisé pour :

— **Log des paramètres** : backbone, batch size, learning rate, gamma, etc.

— **Log des métriques** : loss, accuracy, AUC, kappa à chaque epoch.

— **Log des artefacts** : modèle sauvegardé, matrices de confusion, courbes ROC.

— **Comparaison de runs** : interface web pour comparer les expériences côte à côte.

Lancer l'interface MLflow

`docker compose up mlflow` puis accéder à `http://localhost:5000` pour visualiser toutes les expériences, comparer les métriques et télécharger les modèles.

6 Optimisation du modèle (`src/models/optimize.py`)

L'optimisation post-entraînement vise à réduire la taille et le temps d'inférence du modèle pour le déploiement en production.

6.1 Pruning structurel

Le pruning met à zéro les poids de faible magnitude (les connexions les moins importantes du réseau). Le pipeline utilise un pruning **progressif** : le taux de sparsité augmente graduellement de 0% à 50% au cours du fine-tuning, laissant au réseau le temps de compenser les connexions supprimées.

Métrique	Avant pruning	Après pruning (50%)
Taille du modèle	85 MB	∼45 MB
Paramètres non-nuls	100%	50%
Perte de performance	—	$< 1\%$ de QWK

6.2 Quantification

La quantification réduit la précision numérique des poids. Trois modes sont disponibles :

Dynamic (INT8) Les poids sont stockés en INT8, les activations sont quantifiées dynamiquement à l'exécution. Bon compromis qualité/performance.

Float16 Les poids passent de 32 bits à 16 bits. Réduction de taille $\times 2$ avec une perte de précision négligeable.

Full INT8 Tout est en INT8 (poids et activations). Nécessite un dataset de calibration. Réduction $\times 4$ et accélération maximale sur CPU.

6.3 Test-Time Augmentation (TTA)

Le TTA applique 8 transformations géométriques (rotations + flips) à chaque image à l'inférence et moyenne les prédictions. Cela améliore la robustesse (+2-3% d'accuracy) au prix d'un temps d'inférence $\times 8$.

Le TTA fournit aussi une estimation de l'**incertitude** : si les 8 prédictions divergent fortement, c'est un signal que le cas est ambigu et nécessite une attention clinique particulière.

6.4 Export ONNX

L'export au format ONNX (Open Neural Network Exchange) permet le déploiement sur des runtimes optimisés comme ONNX Runtime ou TensorRT, qui offrent des optimisations supplémentaires (fusion de couches, parallélisation).

7 Évaluation et explicabilité (`src/evaluation/`)

7.1 Métriques complètes

Le module d'évaluation calcule un ensemble exhaustif de métriques :

Accuracy Proportion globale de classifications correctes.

Quadratic Weighted Kappa Métrique de référence pour la classification ordinale. Pénalise les erreurs proportionnellement à la distance entre grades. $\kappa = 1$: accord parfait, $\kappa = 0$: accord aléatoire.

AUC-ROC macro Aire sous la courbe ROC, moyennée sur les 5 classes. Mesure la capacité discriminante indépendamment du seuil de décision.

Sensibilité "referable" Taux de détection des cas nécessitant un référencement (grade ≥ 2). Métrique clinique prioritaire.

Spécificité "referable" Taux de non-référencement correct des cas sains (grade < 2). Mesure le taux de faux positifs évités.

Précision, Rappel, F1 par classe Détail des performances pour chaque grade.

7.2 Résultats obtenus

Métrique	Baseline	Optimisé	Gain
Accuracy	0.73	**0.82**	+12.3%
QWK	0.78	**0.87**	+11.5%
AUC macro	0.85	**0.92**	+8.2%
Sensibilité (réf.)	0.71	**0.89**	+25.4%
Spécificité (réf.)	0.88	**0.93**	+5.7%
Inférence (GPU)	45 ms	**28 ms**	−37.8%

7.3 Explicabilité : Grad-CAM

Grad-CAM (Gradient-weighted Class Activation Mapping) génère des cartes de chaleur montrant les régions de l'image qui contribuent le plus à la décision du modèle. Le processus est le suivant :

1. Les gradients de la classe prédite sont calculés par rapport aux feature maps de la dernière couche convolutive.
2. Ces gradients sont moyennés spatialement pour obtenir un poids d'importance par canal.
3. Les feature maps sont combinées linéairement avec ces poids.
4. Une ReLU ne conserve que les contributions positives.
5. Le résultat est superposé à l'image originale sous forme de heatmap colorée.

Utilité clinique du Grad-CAM

Un radiologue peut vérifier en un coup d'œil si le modèle a identifié les bonnes zones (microanévrismes, exsudats, néovascularisation) ou s'il s'est appuyé sur des artefacts. C'est un outil de **confiance** et de **triage** essentiel pour l'adoption clinique.

7.4 Calibration de confiance (ECE)

L'ECE (Expected Calibration Error) mesure si les scores de confiance du modèle sont fiables. Un modèle bien calibré qui dit "80% de confiance" devrait avoir raison 80% du temps. Le module calcule l'ECE, identifie les bins problématiques et permet la calibration post-hoc si nécessaire.

7.5 Seuils de triage clinique

Le module détermine automatiquement les seuils de décision optimaux pour le triage :

Seuil	Signification
Referable (≥ 2)	Si $P(\text{grade} \geq 2) > \text{seuil} \rightarrow$ le patient est référé à l'ophtalmologue. Le seuil est optimisé pour maintenir une sensibilité $\geq 90\%$.
Sight-threatening (≥ 3)	Si $P(\text{grade} \geq 3) > \text{seuil} \rightarrow$ référencement en urgence.

7.6 Analyse des erreurs

Le module identifie automatiquement les patterns d'erreurs les plus dangereux cliniquement :

- Les confusions les plus fréquentes entre classes.
- Les erreurs à **haute confiance** (>80%) — les plus dangereuses car le modèle est confiant et faux.
- Les **sous-estimations critiques** (le modèle prédit un grade trop bas de ≥ 2 niveaux).

8 API d'inférence (`src/api/`)

8.1 Endpoints FastAPI

L'API REST expose 4 endpoints :

Méthode	Route	Description
`GET`	`/health`	Vérification de l'état du service et du chargement du modèle.
`GET`	`/model/info`	Informations sur le modèle déployé (taille, input, classes).
`POST`	`/predict`	Prédiction sur une image unique. Retourne grade, confiance, probabilités et recommandation clinique.
`POST`	`/predict/batch`	Prédiction sur un batch de 1 à 10 images.

8.2 Format de réponse

Chaque prédiction retourne un objet JSON structuré contenant :

- `grade` : entier 0-4.
- `grade_name` : nom lisible ("No DR", "Mild", etc.).
- `confidence` : score de confiance de la classe prédite.
- `probabilities` : distribution complète sur les 5 classes.
- `clinical_action` : recommandation de suivi ("contrôle annuel", "référer en urgence", etc.).
- `inference_time_ms` : temps de traitement en millisecondes.

8.3 Modèle TFLite en production

En production, l'API utilise le modèle quantifié TFLite (et non le modèle Keras complet). Cela réduit la consommation mémoire, accélère l'inférence et permet un déploiement sur des machines modestes (pas de GPU requis).

9 Déploiement Docker

9.1 Dockerfile multi-stage

Le Dockerfile utilise un build en 2 étapes :

Stage 1 — Builder Installe toutes les dépendances Python dans un répertoire dédié.

Stage 2 — Production Copie uniquement les dépendances installées et le code source dans une image minimale basée sur `python:3.10-slim`. Résultat : une image de production légère sans les outils de build.

9.2 Docker Compose

Le fichier `docker-compose.yml` définit deux services :

1. **api** : L'API FastAPI d'inférence (port 8000), avec le modèle optimisé monté en volume read-only et une limite mémoire de 2 Go.
2. **mlflow** : Le serveur de tracking MLflow (port 5000), avec un backend SQLite pour la persistance des expériences.

10 Tests et qualité de code

10.1 Tests unitaires

Le projet inclut deux suites de tests couvrant les composants critiques :

10.1.1 `test_preprocessing.py`

Vérifie chaque étape du preprocessing :

- Dimensions de sortie correctes pour différentes tailles d'entrée.
- Plage de valeurs ($[0, 1]$ après normalisation).
- Type de données (float32).
- Déterminisme (deux appels identiques donnent le même résultat).
- Gestion des cas limites (chemin invalide, image carrée, etc.).
- Désactivation correcte des étapes optionnelles.

10.1.2 `test_model.py`

Vérifie l'architecture et les fonctions de perte :

- Le modèle se construit sans erreur et produit la bonne forme de sortie.
- Les sorties sont des probabilités valides (≥ 0, somme $= 1$).
- Le backbone est correctement gelé en phase 1 et dégelé en phase 2.
- La Focal Loss avec $\gamma = 0$ se comporte comme la Cross-Entropy.
- Les exemples bien classés ont une loss plus faible que les mal classés.
- Le QWK Loss retourne ≈ 0 pour un accord parfait.

10.2 Commandes Makefile

Le Makefile centralise toutes les commandes courantes :

Commande	Description
`make install`	Installation des dépendances de production
`make test`	Exécution des tests unitaires
`make test-cov`	Tests avec couverture de code HTML
`make lint`	Vérification du code (ruff + mypy)
`make preprocess`	Lancement du pipeline de prétraitement
`make train`	Entraînement du modèle
`make optimize`	Optimisation post-entraînement
`make evaluate`	Évaluation complète avec rapport
`make serve`	Démarrage de l'API via Docker
`make mlflow`	Lancement du serveur MLflow

11 Configuration et utilitaires

11.1 Fichier de configuration YAML

Tous les hyperparamètres sont centralisés dans `configs/train_config.yaml`. Le module `src/utils/config.` gère le chargement avec un système de priorité en 4 niveaux : (1) valeurs par défaut intégrées, (2) fichier YAML, (3) variables d'environnement (préfixe `RETINAI_`), (4) overrides passés en argument. Des validations automatiques vérifient les plages de valeurs.

11.2 Logging structuré

Le module `src/utils/logger.py` configure un double système de logging :

- **Console** : sortie colorée (niveaux différenciés visuellement) pour le développement.

— **Fichier** : format JSON Lines (`.jsonl`) pour l'ingestion dans des outils d'observabilité (ELK, Datadog, etc.).

Les librairies tierces bruyantes (TensorFlow, h5py, urllib3) sont automatiquement réduites au niveau WARNING.

Fin du document — Documentation technique détaillée